Today 8:52 AM

main(1).py
Python

requirements(1).txt
Document

catch(1).py
Python

fish(1).py
Python

game_engine(1).py
Python

hook(1).py
Python

renderer(1).py
Python

I am working on a Pygame Fishing game for a lab assignment for SE.

I have uploaded the complete starter project files, including:
main.py
game/game_engine.py
game/hook.py
game/fish.py
game/catch.py
game/renderer.py

Show more

Python Run

```python
def check_catch(hook, fish_list):
    """
    Returns the fish the hook has caught, or None.
    """
    hook_rect = hook.get_rect()

    for fish in fish_list:
        if hook_rect.colliderect(fish.get_rect()):
```

```
            return fish

    return None
```

Change: `check_catch()` now uses `pygame.Rect.colliderect()` on the existing `get_rect()` methods, so the hook and fish must actually overlap in **both X and Y** to register a catch. catch

Nothing in `Hook`, `Fish`, `GameEngine`, or the other tasks needs to be changed. Their existing `get_rect()` methods already provide the rectangles needed for the collision check. hook fish

Sources

Today 7:39 PM

i did the changes , completed task 1, and pushed it to git, now
Task 2: Multiple fish types

Add at least two fish types to the existing Pygame fishing game. Each fish type should have a different movement speed, point value, and visual appearance (color and/or size). Make sure the correct point value is awarded when each fish type is caught.

Implement **only Task 2** from the README. Do not

Show more

For **Task 2 only**, you only need to change `game/game_engine.py`.

The existing `Fish` class already supports different `speed`, `width`, `height`, `point_value`, and `color`, so there is no need to modify `fish.py`. fish

`game/game_engine.py` — change only `self.fish_list`
Replace the current `self.fish_list` with:

Python Run

```python
self.fish_list = [
    # Small, fast fish — 10 points
    Fish(
        x=100, y=180,
        speed=3,
        width=36, height=18,
        point_value=10,
        color=(80, 180, 220)
    ),

    # Large, slow fish — 20 points
    Fish(
        x=400, y=280,
        speed=-1,
        width=50, height=26,
        point_value=20,
        color=(220, 100, 80)
    ),

    # Medium, faster fish — 30 points
    Fish(
        x=250, y=380,
        speed=4,
        width=30, height=15,
        point_value=30,
        color=(100, 220, 120)
    ),
]
```

So you now have **3 visually and behaviorally different fish types**:

Fish	Speed	Size	Color	Points
Small blue	3	36×18	Blue	10
Large red	-1	50×26	Red	20
Small green	4	30×15	Green	30

The existing `GameEngine` already awards `self.hooked_fish.point_value` when the caught fish reaches the surface, so the correct score is automatically given based on which fish was caught. game_engine

Nothing else needs to be changed. This does not touch Task 3's player-controlled casting or Task 4's timer.

Sources

task 2 completed
Task 3: Player-controlled casting

Implement Task 3 from the README only.

Change the fishing game so that the hook starts in an idle state and a player key, such as SPACE, starts a cast.

The hook should cast only when it is idle. If the player presses SPACE while the hook is already casting or returning, the current cast must not be interrupted or

Show more

For **Task 3 only**, two files need changes: `main.py` and `game/game_engine.py`.

The important point is that the existing `Hook` class already has the required states (`IDLE`, `CASTING`, `RETRACTING`) and `start_cast()` method, so `hook.py` **does not need to change.** hook +1

1. `game/game_engine.py`

Remove the automatic casting from `update()`.

Remove:

Python Run

```python
if self.hook.state == IDLE:
    self.hook.start_cast()
```

So the beginning of `update()` becomes:

```
    def update(self):
        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y
            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                self.hooked_fish = None
        else:
            caught = check_catch(self.hook, self.fish_list)
            if caught is not None:
                self.fish_list.remove(caught)
                self.hooked_fish = caught
                self.hooked_fish.x = self.hook.x
                self.hooked_fish.y = self.hook.y
                self.hook.catch_fish()
```

The existing `update()` currently starts a cast automatically whenever the hook is idle, which is exactly what Task 3 needs to remove. game_engine

2. `main.py`

Change the event loop so **SPACE starts a cast only when the hook is idle**.

Replace this:

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
```

Python Run

with:

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

    elif event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
```

```
        if engine.hook.state == IDLE:
            engine.hook.start_cast()
```

Also add `IDLE` to the import:

```
Python
Run
```

So the imports become:

```
import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE
from game.hook import IDLE
```

Your existing main loop already processes keyboard/window events before calling `engine.update()`, so this fits directly into the current structure. main

Result

- Game starts with hook **idle**.
- Press **SPACE** → cast starts.
- Press SPACE while casting → **nothing happens**.
- Press SPACE while returning → **nothing happens**.
- Hook reaches surface → becomes `IDLE` automatically. hook
- Press SPACE again → new cast starts.
- **Task 4 is untouched.**
- `hook.py`, `fish.py`, `catch.py`, and `renderer.py` remain unchanged.

Sources

task 3 completed
Task 4: 30-second timer

Implement Task 4 from the README only.

Add a 30-second countdown timer that is visible on screen.

When the timer reaches 0:
- Stop allowing fish catches.

Show more

Worked for 11s

For **Task 4 only**, change these **3 files**:

- `game/game_engine.py` — timer, game-over state, restart state
- `main.py` — handle `R` to restart
- `game/renderer.py` — display timer and Game Over screen

`hook.py`, `fish.py`, and `catch.py` stay unchanged, so Tasks 1–3 remain intact. Your Task 1 collision logic continues to be used through `check_catch()`. catch

1. `game/game_engine.py`

Add this import

Python

```python
import pygame
```

Keep your existing imports too:

Python Run

```python
from game.hook import Hook, IDLE
from game.fish import Fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y
```

Replace `__init__()` with:

Python

```python
def __init__(self):
    self.hook = Hook(
        x=WIDTH / 2,
        surface_y=SURFACE_Y,
        max_depth_y=MAX_DEPTH_Y,
        speed=5
    )

    self.fish_list = [
        Fish(
            x=100, y=180,
            speed=3,
            width=36, height=18,
            point_value=10,
            color=(80, 180, 220)
        ),
        Fish(
            x=400, y=280,
            speed=-1,
            width=50, height=26,
            point_value=20,
            color=(220, 100, 80)
        ),
        Fish(
            x=250, y=380,
            speed=4,
            width=30, height=15,
            point_value=30,
            color=(100, 220, 120)
        ),
    ]

    self.hooked_fish = None
    self.score = 0

    self.round_duration = 30
    self.start_time = pygame.time.get_ticks()
    self.game_over = False
```

Replace `update()` with:

Python Run

```python
def update(self):
    if self.game_over:
```

```
            return

        elapsed = (pygame.time.get_ticks() - self.start_time) / 1000
        self.time_left = max(0, self.round_duration - elapsed)

        if self.time_left <= 0:
            self.game_over = True
            return

        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y

            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                self.hooked_fish = None
        else:
            caught = check_catch(self.hook, self.fish_list)

            if caught is not None:
                self.fish_list.remove(caught)
                self.hooked_fish = caught
                self.hooked_fish.x = self.hook.x
                self.hooked_fish.y = self.hook.y
                self.hook.catch_fish()
```

The important part is that the timer check happens **before** `check_catch()`, so once time reaches 0, no more fish can be caught.

Add this new method to `GameEngine`:

Python Run

```
def restart(self):
    self.hook.state = IDLE
    self.hook.y = self.hook.surface_y

    self.fish_list = [
        Fish(
            x=100, y=180,
            speed=3,
            width=36, height=18,
```

```
            point_value=10,
            color=(80, 180, 220)
        ),
        Fish(
            x=400, y=280,
            speed=-1,
            width=50, height=26,
            point_value=20,
            color=(220, 100, 80)
        ),
        Fish(
            x=250, y=380,
            speed=4,
            width=30, height=15,
            point_value=30,
            color=(100, 220, 120)
        ),
    ]

    self.hooked_fish = None
    self.score = 0
    self.start_time = pygame.time.get_ticks()
    self.time_left = self.round_duration
    self.game_over = False
```

This resets the round completely while preserving your Task 2 fish types.

Replace `draw()` with:

Python Run

```
def draw(self, surface, font):
    from game import renderer

    draw_list = list(self.fish_list)

    if self.hooked_fish is not None:
        draw_list.append(self.hooked_fish)

    renderer.draw_scene(surface, self.hook, draw_list)

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )
```

```
        renderer.draw_text(
            surface,
            font,
            f"Time: {max(0, int(self.time_left))}",
            (560, 10)
        )

        if self.game_over:
            renderer.draw_game_over(surface, font, self.score)
```

2. `main.py`

You already have the Task 3 SPACE logic.

Keep it, and add the **R** handling.

Your event loop should become:

Python

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_SPACE:
            if engine.hook.state == IDLE:
                engine.hook.start_cast()

        elif event.key == pygame.K_r:
            if engine.game_over:
                engine.restart()
```

Keep this import from Task 3:

Python Run

```
from game.hook import IDLE
```

So **SPACE still only starts a cast when idle**, and **R only works after Game Over**. The existing hook state behavior supports this directly. hook

3. `game/renderer.py`

Add this function at the end of the file:

Python Run

```python
def draw_game_over(surface, font, score):
    overlay = pygame.Surface(surface.get_size(), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 170))
    surface.blit(overlay, (0, 0))

    game_over_text = font.render("GAME OVER", True, (255, 255, 255))
    score_text = font.render(f"Final Score: {score}", True, (255, 255, 255))
    restart_text = font.render("Press R to restart", True, (255, 255, 255))

    center_x = surface.get_width() // 2

    surface.blit(
        game_over_text,
        game_over_text.get_rect(center=(center_x, 190))
    )

    surface.blit(
        score_text,
        score_text.get_rect(center=(center_x, 230))
    )

    surface.blit(
        restart_text,
        restart_text.get_rect(center=(center_x, 270))
    )
```

Final Task 4 behavior

1. Game starts with **30 seconds**.
2. Timer is visible at the top-right.
3. SPACE continues to work exactly as in **Task 3**.
4. Fish catching continues to use your **Task 1 rectangle collision**.
5. Fish retain the different **Task 2 scores/types**.
6. At `0` seconds:
   - catching stops
   - game updates stop
   - **GAME OVER** appears

- final score appears
- `R` becomes available

7. Press `R`:
   - timer → **30**
   - score → **0**
   - fish are restored
   - hook → **IDLE**
   - game becomes playable again.

No Task 5+ functionality or unrelated changes are needed.

Sources

Task 5: Continuous fish spawning

The original README Tasks 1–4 are already completed. Do not change or break any of the existing functionality.

I want to add one extra feature as Task 5:

- Fish should continue appearing throughout the 30-second round.
- When the player successfully catches a fish, that fish should be replaced by a new fish.

Show more

For **Task 5 only**, you need to modify **just one file**:

`game/game_engine.py`

Why: All fish creation, the fish list, catching logic, scoring, timer state, and restart logic are already handled by `GameEngine`. game_engine

The changes will be:

1. **Add `random` import**
   - Used to randomly choose between the existing blue, red, and green fish types.
   - Used to generate a suitable random `(x, y)` position in the water.

2. **Add a fish-spawning helper**
   - Creates a new fish using one of the **three existing Task 2 fish configurations**.
   - Preserves each type's:
     - speed
     - point value
     - size
     - color
3. **Modify the catch section in `update()`**
   - Currently, when a fish is caught, it is removed from `fish_list` and stored as `hooked_fish`. game_engine
   - After that caught fish is successfully brought back to the surface and its points are awarded, a **new random fish will be added** to `fish_list`.
   - This means the number of available fish stays replenished and the score can go beyond the original 60.
4. **Keep spawning disabled after Game Over**
   - The existing `game_over` check at the beginning of `update()` already stops game logic once the timer reaches 0. game_engine
   - We will keep that behavior intact.
5. **Update `restart()`**
   - Restart will restore the normal starting fish and reset the timer/score as it already does.
   - No changes are needed to the restart mechanism beyond making sure it uses the existing fish types.

Files that will NOT be modified

- `main.py` — **no change**; SPACE/R controls from Task 3/4 remain intact.
- `game/hook.py` — **no change**.
- `game/fish.py` — **no change**; it already accepts different speed, size, point value, and color. fish
- `game/catch.py` — **no change**; your Task 1 rectangle-overlap detection stays intact. catch
- `game/renderer.py` — **no change**; Task 4 timer and Game Over display remain as they are.

So the exact answer is:

Modify only `game/game_engine.py`.

No code changes yet, as requested.

Sources

Okay, now give me the complete updated `game/game_engine.py` code so I can replace the entire file. Keep all Task 1–4 functionality unchanged.

Yes. Replace the entire `game/game_engine.py` with this version. It keeps Tasks 1–4 and adds Task 5 continuous spawning.

Python Run

```python
import random
import pygame

from game.hook import Hook, IDLE
from game.fish import Fish
from game.catch import check_catch
from game.renderer import WIDTH, HEIGHT, SURFACE_Y, MAX_DEPTH_Y


class GameEngine:
    def __init__(self):
        self.hook = Hook(
            x=WIDTH / 2,
            surface_y=SURFACE_Y,
            max_depth_y=MAX_DEPTH_Y,
            speed=5
        )

        self.fish_list = [
            Fish(
                x=100, y=180,
                speed=3,
                width=36, height=18,
                point_value=10,
                color=(80, 180, 220)
            ),
            Fish(
```

```
                x=400, y=280,
                speed=-1,
                width=50, height=26,
                point_value=20,
                color=(220, 100, 80)
            ),
            Fish(
                x=250, y=380,
                speed=4,
                width=30, height=15,
                point_value=30,
                color=(100, 220, 120)
            ),
        ]

        self.hooked_fish = None
        self.score = 0

        self.round_duration = 30
        self.start_time = pygame.time.get_ticks()
        self.time_left = self.round_duration
        self.game_over = False

    def create_random_fish(self):
        fish_types = [
            {
                "speed": 3,
                "width": 36,
                "height": 18,
                "point_value": 10,
                "color": (80, 180, 220)
            },
            {
                "speed": -1,
                "width": 50,
                "height": 26,
                "point_value": 20,
                "color": (220, 100, 80)
            },
            {
                "speed": 4,
                "width": 30,
                "height": 15,
                "point_value": 30,
                "color": (100, 220, 120)
            }
        ]

        fish_type = random.choice(fish_types)
```

```
        x = random.randint(40, WIDTH - 40)
        y = random.randint(SURFACE_Y + 50, MAX_DEPTH_Y - 20)

        return Fish(
            x=x,
            y=y,
            speed=fish_type["speed"],
            width=fish_type["width"],
            height=fish_type["height"],
            point_value=fish_type["point_value"],
            color=fish_type["color"]
        )

    def update(self):
        if self.game_over:
            return

        elapsed = (pygame.time.get_ticks() - self.start_time) / 1000
        self.time_left = max(0, self.round_duration - elapsed)

        if self.time_left <= 0:
            self.game_over = True
            return

        self.hook.update()

        for fish in self.fish_list:
            fish.update(WIDTH)

        if self.hooked_fish is not None:
            self.hooked_fish.x = self.hook.x
            self.hooked_fish.y = self.hook.y

            if self.hook.state == IDLE:
                self.score += self.hooked_fish.point_value
                self.hooked_fish = None

                # Replace the caught fish with a new random fish.
                self.fish_list.append(self.create_random_fish())

        else:
            caught = check_catch(self.hook, self.fish_list)

            if caught is not None:
                self.fish_list.remove(caught)
                self.hooked_fish = caught

                self.hooked_fish.x = self.hook.x
```

```
                self.hooked_fish.y = self.hook.y

                self.hook.catch_fish()

    def restart(self):
        self.hook.state = IDLE
        self.hook.y = self.hook.surface_y

        self.fish_list = [
            Fish(
                x=100, y=180,
                speed=3,
                width=36, height=18,
                point_value=10,
                color=(80, 180, 220)
            ),
            Fish(
                x=400, y=280,
                speed=-1,
                width=50, height=26,
                point_value=20,
                color=(220, 100, 80)
            ),
            Fish(
                x=250, y=380,
                speed=4,
                width=30, height=15,
                point_value=30,
                color=(100, 220, 120)
            ),
        ]

        self.hooked_fish = None
        self.score = 0
        self.start_time = pygame.time.get_ticks()
        self.time_left = self.round_duration
        self.game_over = False

    def draw(self, surface, font):
        from game import renderer

        draw_list = list(self.fish_list)

        if self.hooked_fish is not None:
            draw_list.append(self.hooked_fish)

        renderer.draw_scene(surface, self.hook, draw_list)

        renderer.draw_text(
```

```
            surface,
            font,
            f"Score: {self.score}",
            (10, 10)
        )

        renderer.draw_text(
            surface,
            font,
            f"Time: {max(0, int(self.time_left))}",
            (560, 10)
        )

        if self.game_over:
            renderer.draw_game_over(surface, font, self.score)
```

What changed for Task 5

The only new behavior is:

Python Run

```
self.fish_list.append(self.create_random_fish())
```

after the caught fish reaches the surface and its points are awarded.

So if you catch the **30-point green fish**, you get 30 points and a new randomly selected fish appears. Therefore the score is no longer capped at 60.

Your existing Task 1 `check_catch()` is still used unchanged, so the fish must actually overlap the hook. catch

Your Task 2 fish properties are preserved, and Task 3/4's hook controls, timer, Game Over, and restart behavior remain in place. The existing `Fish` class supports the individual speed, size, point value, and color properties used here. fish

Sources

ChatGPT can make mistakes. Check important info.